

Lecture 02: Numbers and Mutable Variables

Introduction to Programming with Kotlin

Agenda

1. The integer type `Int`
2. Integer division and remainder
3. Mutable variables with `var`
4. Floating-point numbers
5. Long, Short, and Byte

What is `Int`?

`Int` is Kotlin's default 32-bit signed integer type.

- Range: about $-2 \cdot 10^9$ to $2 \cdot 10^9$ (exactly $-2^{31} \dots 2^{31} - 1$)
- Whole numbers only: 0, -3, 42, 1000
- Integer literals like 42 have type `Int` by default

When you need `Int`

Counts, ages, indices, years, loop counters, simple arithmetic on wholes.

Declaring and using `Int`

```
1 fun main() {  
2     val a: Int = 10  
3     val b = 3           // inferred as Int  
4     println(a + b)      // 13  
5     println(a - b)      // 7  
6     println(a * b)      // 30  
7 }
```

Arithmetic operators: `+`, `-`, `*`, `/`, `%`

Reading an `Int`

```
1 fun main() {  
2     print("Enter an integer: ")  
3     val n = readln().toInt()  
4     println("You entered $n")  
5     println("Double is ${n * 2}")  
6 }
```

Common mistake

Forgetting `toInt()`. `readln()` returns `String`; `"10" + "20"` is `"1020"`, not `30`.

Quiz

What is the type of the literal 100 in Kotlin?

1. Long
2. Int
3. Double
4. String

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 1

Answer

2. `Int`

Unsuffixed integer literals are `Int`. Write `100L` for Long, or `100.0` for Double.

Integer division

When both operands are integers, `/` performs **integer division**: the fractional part is discarded (truncated toward zero).

- $7 / 2 \rightarrow 3$ (not 3.5)
- $(-7) / 2 \rightarrow -3$
- Remainder: $7 \% 2 \rightarrow 1$

Division and remainder

```
1 fun main() {  
2     val total = 17  
3     val group = 5  
4     println(total / group)    // 3  
5     println(total % group)    // 2  
6     // 3 groups of 5, with 2 left over  
7 }
```

Check: $(total / group) * group + (total \% group) = total$ (for non-negative values).

Getting a fractional result

Convert at least one operand to floating-point:

```
1 fun main() {  
2     val a = 7  
3     val b = 2  
4     println(a / b)           // 3    (Int)  
5     println(a.toDouble() / b) // 3.5  
6     println(7.0 / 2)         // 3.5  
7 }
```

Try this

Read two integers and print both the integer quotient and the exact ratio as a decimal.

Quiz 2 — Integer division

Quiz

What does `println(20 / 3)` print?

1. 6.666...
2. 6
3. 7
4. 20 / 3

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 2

Answer

2. 6

Both operands are `Int`, so the result is truncated integer division. Remainder would be $20 \% 3 = 2$.

val (immutable binding)

- Assign once
- Cannot reassign
- Prefer by default

var (mutable binding)

- Can reassign
- Same name, new value
- Use when the value must change

“Immutable binding” means the *variable* cannot be pointed at a new value. (Contents of a mutable list can still change — later.)

Updating with `var`

```
1 fun main() {  
2     var score = 0  
3     score = score + 10  
4     score += 5           // same as score = score + 5  
5     println(score)      // 15  
6  
7     var lives = 3  
8     lives -= 1  
9     println(lives)      // 2  
10 }
```

Compound assignments: `+=`, `-=`, `*=`, `/=`, `%=`

When `val` is enough

```
1 fun main() {  
2     val a = readln().toInt()  
3     val b = readln().toInt()  
4     val sum = a + b           // no need for var  
5     println(sum)  
6 }
```

Common mistake

Using `var` everywhere “just in case”. Start with `val`; switch to `var` only when reassignment is required.

Quiz 3 — Mutable variables

Quiz

Which snippet compiles?

1. `val x = 1; x = 2`
2. `var x = 1; x = 2`
3. `val x; x = 1`
4. `var x = 1; x = "hi"`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 3

Answer

2. `var x = 1; x = 2`

(1) cannot reassign `val`. (3) needs a type or initializer. (4) type is fixed as `Int`; cannot assign a `String`.

Floating-point types

For numbers with a fractional part, use floating-point types:

Type	Size	Typical use
Double	64-bit	default decimal type
Float	32-bit	less precision; rarer in intro code

Literals: `3.14` is `Double`; `3.14f` is `Float`.

Working with Double

```
1 fun main() {  
2     val price = 19.99  
3     val taxRate = 0.08  
4     val total = price * (1 + taxRate)  
5     println("Total: $total")  
6  
7     print("Enter a decimal: ")  
8     val x = readln().toDouble()  
9     println("Half is ${x / 2}")  
10 }
```

Precision warning

Floating-point arithmetic is approximate in binary.

- $0.1 + 0.2$ may not be exactly 0.3
- Fine for science demos and many apps; careful with money
- For currency, later courses use integer cents or decimal libraries

Try this

Print $0.1 + 0.2$ and discuss what you see.

Quiz 4 — Floating-point

Quiz

How do you read a decimal number from the console into a Double?

1. `val x = readln()`
2. `val x = readln().toInt()`
3. `val x = readln().toDouble()`
4. `val x = Double(readln())`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

3. `val x = readln().toDouble()`

Parse the input string with `toDouble()`. `toInt()` would reject values like "3.5".

Other integer types

Type	Bits	Approx. range	Literal tip
Byte	8	$-128 \dots 127$	convert explicitly
Short	16	$\approx \pm 32\,000$	convert explicitly
Int	32	$\approx \pm 2 \cdot 10^9$	default
Long	64	very large	suffix L

enough.

For most intro programs, Int and Long are

```
1 fun main() {  
2     val population = 8_000_000_000L    // underscores OK  
3     val big = 2_000_000_000  
4     // big * 3 may overflow Int; use Long:  
5     val safer = big.toLong() * 3  
6     println(population)  
7     println(safer)  
8 }
```

toLong(), toShort(), toByte() convert when needed. Converting out of range can wrap or throw depending on API — stay in range.

Byte and Short

```
1 fun main() {  
2     val small: Byte = 100  
3     val medium: Short = 30_000  
4     // Arithmetic often promotes to Int:  
5     val sum = small + medium    // Int  
6     println(sum)  
7 }
```

You rarely declare Byte/Short unless matching a file format, network protocol, or memory-tight structure.

Quiz 5 — Integer sizes

Quiz

Which literal is a Long?

1. 42
2. 42.0
3. 42L
4. "42"

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

3. 42L

The L suffix marks a Long literal. 42 is Int; 42.0 is Double.

Summary

- `Int` — default whole-number type
- `Integer` / truncates; use `%` for remainder
- `var` when you must reassign; prefer `val` otherwise
- `Double` for decimals; watch floating-point precision
- `Long` / `Short` / `Byte` for other integer widths

Looking ahead

Next: conditional operators (`if/else`), combining conditions, and `if/when` as expressions.